

Python
for DA

Summary Session 11



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Что такое создание массивов?



Создание массивов

Это фундаментальный аспект использования NumPy, поскольку массивы являются основной структурой данных, используемой для численных вычислений.

Экспресс-опрос

?

Что такое np.array()?



`np.array()`

Это функция, которая используется для создания массива в NumPy. Она преобразует список, кортеж или другой объект, похожий на последовательность, в массив NumPy.

Функция np.array()

```
numpy.array(object, dtype=None, copy=True, order='K', ndmin=0)
```

Пример np.array()

```
import numpy as np

# Создание одномерного массива из списка
list_data = [1, 2, 3, 4, 5]
array_from_list = np.array(list_data)
print("Массив из списка:", array_from_list)

# Создание одномерного массива из кортежа
tuple_data = (6, 7, 8, 9, 10)
array_from_tuple = np.array(tuple_data)
print("Массив из кортежа:", array_from_tuple)

# Создание двумерного массива из списка списков
list_of_lists = [[1, 2, 3], [4, 5, 6]]
array_2d = np.array(list_of_lists)
print("Двумерный массив из списка списков:\n", array_2d)
```

Атрибуты массивов в NumPy

- ☒ Атрибут ndim
- ☒ Атрибут shape
- ☒ Атрибут size
- ☒ Атрибут dtype
- ☒ Атрибут itemsize

Пример атрибута ndim

```
import numpy as np

# Создание одномерного массива
array1 = np.array([1, 2, 3])
print(array1.ndim) # Вывод: 1

# Создание двумерного массива
array2 = np.array([[2, 3, 4], [5, 6, 7]])
print(array2.ndim) # Вывод: 2
```

Пример атрибут shape

```
import numpy as np

# Создание одномерного массива
array1 = np.array([1, 2, 3])
print(array1.shape) # Вывод: (3,)
```



```
# Создание двумерного массива
array2 = np.array([[3, 4, 5], [6, 7, 8]])
print(array2.shape) # Вывод: (2, 3)
```

Пример атрибут size

```
import numpy as np

# Создание одномерного массива
array1 = np.array([1, 2, 3])
print(array1.size) # Выходные данные: 3

# Создание двумерного массива
array2 = np.array([[3, 5, 7], [8, 9, 10]])
print(array2.size) # Вывод: 6
```

Пример атрибут dtype

```
import numpy as np

# Создание массива целых чисел
array1 = np.array([10, 20, 30])
print(array1.dtype) # Выходные данные: int64

# Создание массива строк
array2 = np.array(["Python", "NumPy", "Data"])
print(array2.dtype) # Вывод: <U6
```

Пример атрибут `itemsize`

```
import numpy as np

# Создание массива 64-битных целых чисел
array1 = np.array([1, 2, 3], dtype=np.int64)
print(array1.itemsize) # Выходные данные: 8

# Создание массива 32-битных целых чисел
array2 = np.array([1, 2, 3], dtype=np.int32)
print(array2.itemsize) # Вывод: 4
```


Функции для инициализации массивов



`np.zeros()`



`np.ones()`



`np.full()`

```
import numpy as np

# Создание одномерного массива из списка
list_data = [1, 2, 3, 4, 5]
array_from_list = np.array(list_data)
print("Массив из списка:", array_from_list)

# Создание одномерного массива из кортежа
tuple_data = (6, 7, 8, 9, 10)
array_from_tuple = np.array(tuple_data)
print("Массив из кортежа:", array_from_tuple)

# Создание двумерного массива из списка списков
list_of_lists = [[1, 2, 3], [4, 5, 6]]
array_2d = np.array(list_of_lists)
print("Двумерный массив из списка списков:\n", array_2d)
```

Пример np.zeros()

```
1D массив нулей: [0. 0. 0. 0. 0.]
Двумерный массив нулей:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

Выходные данные

```
import numpy as np

# Создание одномерного массива единиц
array1 = np.ones(5)
print("одномерный массив единиц:", array1)

# Создание двумерного массива единиц
array2 = np.ones((3, 4))
print("Двумерный массив единиц:\n", array2)
```

Пример np.ones()

```
одномерный массив единиц: [1. 1. 1. 1. 1.]
Двумерный массив единиц:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

Выходные данные

```
import numpy as np

# Создание одномерного массива, заполненного значением 7
array1 = np.full(5, 7)
print("Одномерный массив, заполненный значением 7:",
      array1)

# Создание двумерного массива, заполненного значением
3,14
array2 = np.full((3, 4), 3.14)
print("Двумерный массив, заполненный значением 3.14:\n",
      array2)
```

Пример np.full()

```
Одномерный массив, заполненный значением 7: [7 7 7 7 7]
Двумерный массив, заполненный значением 3.14:
[[3.14 3.14 3.14 3.14]
 [3.14 3.14 3.14 3.14]
 [3.14 3.14 3.14 3.14]]
```

Выходные данные

Инициализация массивов: создание последовательностей



`np.arange()`



`np.linspace()`

Пример np.arange()

```
import numpy as np
```

```
# Создание массива со значениями от 0 до 9
```

```
array1 = np.arange(10)
```

```
print("Массив со значениями от 0 до 9:", array1)
```

```
# Создание массива со значениями от 1 до 9
```

```
array2 = np.arange(1, 10)
```

```
print("Массив со значениями от 1 до 9:", array2)
```

```
# Создание массива со значениями от 1 до 9 с шагом 2
```

```
array3 = np.arange(1, 10, 2)
```

```
print("Массив со значениями от 1 до 9 с шагом 2:", array3)
```

```
# Создание массива со значениями с плавающей точкой
```

```
array4 = np.arange(0, 1, 0.1)
```

```
print("Массив со значениями с плавающей точкой от 0 до 1 с шагом 0,1:", array4)
```

Выходные данные

Массив со значениями от 0 до 9: [0 1 2 3 4 5 6 7 8 9]

Массив со значениями от 1 до 9: [1 2 3 4 5 6 7 8 9]

Массив со значениями от 1 до 9 с шагом 2: [1 3 5 7 9]

Массив со значениями с плавающей точкой от 0 до 1 с шагом 0,1: [0. 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]

Пример np.linspace()

```
import numpy as np

# Создание массива с 10 равномерно распределенными значениями от 0 до 1
array1 = np.linspace(0, 1, 10)
print("Массив с 10 равномерно распределенными значениями от 0 до 1:", array1)

# Создание массива с 5 равномерно распределенными значениями от 1 до 10
array2 = np.linspace(1, 10, 5)
print("Массив с 5 равномерно распределенными значениями от 1 до 10:", array2)

# Создание массива с 5 равномерно распределенными значениями от 1 до 10, исключая конечную точку
array3 = np.linspace(1, 10, 5, endpoint=False)
print("Массив с 5 равномерно распределенными значениями от 1 до 10, исключая конечную точку:", array3)

# Создание массива и возвращение размера шага
array4, step = np.linspace(0, 1, 5, retstep=True)
print("Массив с 5 равномерно расположенными значениями от 0 до 1:", array4)
print("Размер шага:", step)
```


Выходные данные

Массив с 10 равномерно распределенными значениями от 0 до 1: [0.01111111 0.22222222 0.33333333 0.44444444 0.55555556
0.66666667 0.77777778 0.88888889 1.]

Массив с 5 равномерно распределенными значениями от 1 до 10: [1. 3.25 5.5 7.75 10.]

Массив с 5 равномерно распределенными значениями от 1 до 10, исключая конечную точку: [1. 2.8 4.6 6.4 8.2]

Массив с 5 равномерно расположенными значениями от 0 до 1: [0. 0.25 0.5 0.75 1.]

Размер шага: 0.25

Экспресс-опрос

?

Что такое Broadcasting?



Broadcasting

Это техника, которая позволяет выполнять арифметические операции над массивами различной формы при определенных условиях.



Поэлементные операции

Это операции, которые применяются к каждому соответствующему элементу массивов. К таким операциям относятся сложение, вычитание, умножение, деление и другие.

Пример сложения

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = a + b
print(result) # Вывод: [5 7 9]

result = np.add(a, b)
print(result) # Вывод: [5 7 9]
```

Пример вычитания

```
result = a - b  
print(result) # Вывод: [-3 -3 -3]
```

```
result = np.subtract(a, b)  
print(result) # Вывод: [-3 -3 -3]
```

Пример умножения

```
result = a * b  
print(result) # Вывод: [ 4 10 18]
```

```
result = np.multiply(a, b)  
print(result) # Вывод: [ 4 10 18]
```

Пример деления

```
result = a / b  
print(result) # Вывод: [0.25 0.4 0.5 ]
```

```
result = np.divide(a, b)  
print(result) # Вывод: [0.25 0.4 0.5 ]
```


Пример возведения в степень

```
result= a ** b  
print(result) # Вывод: [ 1 32 729]
```

```
result = np.power(a, b)  
print(result) # Вывод: [ 1 32 729]
```

Правила broadcasting'a

- Если массивы не имеют одинакового количества измерений, добавляйте 1 к форме меньшего массива до тех пор, пока обе формы не станут одинаковой длины.
- Массивы совместимы для broadcasting'a, если для каждой размерности их размеры либо одинаковы, либо один из них равен 1.
- Результирующая форма массива - это максимальный размер по каждому измерению входных массивов.

```
import numpy as np

# Массив a формы (3, 1)
a = np.array([[1], [2], [3]])

# Массив b формы (1, 4)
b = np.array([4, 5, 6, 7])

# Передача a и b в форму (3, 4)
result = a + b
print(result)
```

Пример broadcasting'a

```
[[ 5  6  7  8]
 [ 6  7  8  9]
 [ 7  8  9 10]]
```

Выходные данные

```
import numpy as np

# Создание двух матриц
matrix1 = np.array([[1, 3], [5, 7]])
matrix2 = np.array([[2, 6], [4, 8]])

# Умножение матриц
result = np.dot(matrix1, matrix2)
print("Матричное умножение:\n", result)
```

Пример умножения матриц

```
Матричное умножение:
[[14 30]
 [38 86]]
```

Вывод

```
import numpy as np

# Создание матрицы
matrix = np.array([[1, 3], [5, 7]])

# Транспонирование матрицы
transpose = np.transpose(matrix)
print("Транспонирование:\n", transpose)
```

Пример транспонирования матрицы

```
Транспонирование:
[[1 5]
 [3 7]]
```

Вывод

Пример изменения формы массивов: np.array.reshape()

```
import numpy as np

# 1D в 2D
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print("От 1D к 2D:\n", newarr)

# 1D в 3D
newarr = arr.reshape(2, 3, 2)
print("От 1D к 3D:\n", newarr)

# Использование -1 для неизвестной размерности
newarr = arr.reshape(2, 2, -1)
print("Использование -1 для неизвестной размерности:\n", newarr)
```

Вывод

От 1D к 2D:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

От 1D к 3D:

```
[[[ 1  2]
 [ 3  4]
 [ 5  6]]
```

```
[[ 7  8]
 [ 9 10]
 [11 12]]]
```

Использование -1 для неизвестной размерности:

```
[[[ 1  2  3]
 [ 4  5  6]]
```

```
[[ 7  8  9]
 [10 11 12]]]
```

Пример изменения размера массивов: np.resize()

```
import numpy as np

# Исходный массив
arr = np.array([[0, 1], [2, 3]])

# Изменение размера массива в большую сторону
resized_arr = np.resize(arr, (2, 3))
print("Изменен размер массива на больший:\n", resized_arr)

# Изменение размера массива в меньшую сторону
resized_arr = np.resize(arr, (1, 4))
print("Изменен размер массива на меньший:\n", resized_arr)
```


Вывод

Изменен размер массива на больший:

```
[[0 1 2]
```

```
[3 0 1]]
```

Изменен размер массива на меньший:

```
[[0 1 2 3]]
```

Пример добавления размеров: np.newaxis

```
import numpy as np

# Исходный одномерный массив
arr = np.array([1, 2, 3])

# Добавление новой оси
newarr = arr[:, np.newaxis]
print("Добавление новой оси с помощью np.newaxis:\n", newarr)
```

Вывод

Добавление новой оси с помощью `pr.newaxis`:

```
[[1]
```

```
[2]
```

```
[3]]
```

Пример генерации случайных чисел: np.random.rand()

```
import numpy as np

# Генерация одного случайного числа
random_number = np.random.rand()
print("Одиночное случайное число:", random_number)

# Генерация одномерного массива случайных чисел
random_array_1d = np.random.rand(5)
print("Одномерный массив случайных чисел:", random_array_1d)

# Генерация двумерного массива случайных чисел
random_array_2d = np.random.rand(3, 4)
print("Двумерный массив случайных чисел:\n", random_array_2d)
```

Выходные данные

Одиночное случайное число: 0.3439993495686171

Одномерный массив случайных чисел: [0.51118773 0.49025769 0.98302143 0.73244607 0.00378275]

Двумерный массив случайных чисел:

[[0.48992219 0.9731271 0.28731655 0.7431044]

[0.01041924 0.29597925 0.57869845 0.29415558]

[0.1650352 0.06879116 0.12179249 0.61111302]]

Пример генерации случайных чисел: np.random.randn()

```
import numpy as np

# Генерация одного случайного числа из стандартного нормального распределения
random_number = np.random.randn()
print("Одиночное случайное число из стандартного нормального распределения:", random_number)

# Генерация одномерного массива случайных чисел из стандартного нормального распределения
random_array_1d = np.random.randn(5)
print("1D массив случайных чисел из стандартного нормального распределения:", random_array_1d)

# Генерирование двумерного массива случайных чисел из стандартного нормального распределения
random_array_2d = np.random.randn(3, 4)
print("Двумерный массив случайных чисел из стандартного нормального распределения:\n",
      random_array_2d)
```

Выходные данные

Одиночное случайное число из стандартного нормального распределения: -0.6687640529109288

1D массив случайных чисел из стандартного нормального распределения: [0.79550431 -0.18718504 0.63239159 -0.95170626 0.9031778]

Двумерный массив случайных чисел из стандартного нормального распределения:

[[-0.99511721 -0.26859096 -1.88186055 -1.22788821]

[-0.2010815 -0.49854496 0.62376896 1.27257252]

[0.49228158 -0.02089429 1.73461492 0.34107269]]

Пример генерации случайных чисел: np.random.randint()

```
import numpy as np

# Генерация одного случайного целого числа
random_int = np.random.randint(10)
print("Одиночное случайное целое число:", random_int)

# Генерация одномерного массива случайных целых чисел
random_array_1d = np.random.randint(1, 10, 5)
print("1D массив случайных целых чисел:", random_array_1d)

# Генерирование двумерного массива случайных целых чисел
random_array_2d = np.random.randint(1, 10, (3, 4))
print("Двумерный массив случайных целых чисел:\n", random_array_2d)
```


Выходные данные

Одиночное случайное целое число: 1

1D массив случайных целых чисел: [7 4 3 4 7]

Двумерный массив случайных целых чисел:

[[9 9 5 1]

[7 9 9 8]

[3 6 2 6]]

Пример случайной выборки: np.random.choice()

```
import numpy as np

# Создание массива
arr = np.array([1, 2, 3, 4, 5])

# Случайный выбор одного элемента
random_choice = np.random.choice(arr)
print("Случайно выбранный элемент:", random_choice)

# Случайный выбор нескольких элементов с заменой
random_choices = np.random.choice(arr, size=3, replace=True)
print("Случайно выбранные элементы с заменой:", random_choices)

# Случайный выбор нескольких элементов без замены
random_choices = np.random.choice(arr, size=3, replace=False)
print("Случайный выбор элементов без замены:", random_choices)

# Случайный выбор элементов с заданными вероятностями
random_choices = np.random.choice(arr, size=3, p=[0.1, 0.2, 0.3, 0.2, 0.2])
print("Случайно выбранные элементы с заданными вероятностями:", random_choices)
```

Выходные данные

Случайно выбранный элемент: 1

Случайно выбранные элементы с заменой: [1 1 4]

Случайный выбор элементов без замены: [3 1 2]

Случайно выбранные элементы с заданными вероятностями: [5 2 2]

Пример воспроизводимости: np.random.seed()

```
import numpy as np

# Установка семени
np.random.seed(42)

# Генерация случайных чисел
random_array_1 = np.random.rand(3)
print("Случайный массив 1:", random_array_1)

# Сброс заправки
np.random.seed(42)

# Снова генерируем случайные числа
random_array_2 = np.random.rand(3)
print("Случайный массив 2:", random_array_2)
```

Выходные данные

Случайный массив 1: [0.37454012 0.95071431 0.73199394]

Случайный массив 2: [0.37454012 0.95071431 0.73199394]



ВОПРОСЫ



Экспресс-опрос

?

Что такое серия Pandas?



Серия Pandas

Это одномерный маркированный массив, способный хранить данные любого типа. Построен поверх массивов NumPy и предоставляет дополнительную функциональность, такую как индексация по меткам и выравнивание данных.

Создание серии Pandas



Непосредственно из списка или массива



С помощью функций NumPy



Из словаря

```
import pandas as pd

data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print(series)
```

Пример: Создание серии из списка

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

Вывод:

```
import numpy as np

# Использование numpy.linspace()
ser1 = pd.Series(np.linspace(3, 33, 3))
print(ser1)

# Использование numpy.random.normal()
ser2 = pd.Series(np.random.normal(0.0, 1.0,
5))
print(ser2)
```

Пример: Создание серии с помощью функций NumPy

```
0      3.0
1     18.0
2     33.0
dtype: float64
0      0.178420
1     -0.548335
2      0.604626
3     -0.058498
4      0.354780
dtype: float64
```

Вывод

```
series1 = pd.Series([1, 2, 3, 4, 5])
series2 = pd.Series([10, 20, 30, 40, 50])

# Сложение
result= series1 + series2
print(result)
```

Пример: Арифметические операции

```
0    11
1    22
2    33
3    44
4    55
dtype: int64
```

Вывод

```
series = pd.Series([1, 2, 3, 4, 5])
```

```
# Сумма
print(series.sum())
```

```
# Среднее
print(series.mean())
```

```
15
3.0
```

Пример: Агрегация

Выходные данные

```
series = pd.Series([10, 20, 30, 40, 50])

# Доступ к первому элементу
print(series[0])

# Нарезка
print(series[1:4])
```

Пример: Позиционное индексирование

```
10
1    20
2    30
3    40
dtype: int64
```

Выходные данные

```
series = pd.Series([10, 20, 30, 40, 50],
index=['a', 'b', 'c', 'd', 'e'])
```

```
# Доступ к элементу с меткой 'b'
print(series['b'])
```

```
# Нарезка с метками
print(series['b':'d'])
```

```
20
b    20
c    30
d    40
dtype: int64
```

Пример: Индексация на основе меток

Выходные данные



Pandas DataFrame

Это двумерная, изменяемая по размеру и потенциально неоднородная табличная структура данных с маркированными осями. Каждый столбец в DataFrame может содержать различные типы данных.


```
import pandas as pd

data = {
    "Name": ["Том", "Ник", "Криш", "Джек"],
    "Возраст": [20, 21, 19, 18]
}
df = pd.DataFrame(data)
print(df)
```

Пример: Создание фрейма данных из словаря

	Name	Возраст
0	Том	20
1	Ник	21
2	Криш	19
3	Джек	18

Вывод

```
data = [['Tom', 20], ['Nick', 21], ['Krish',
19], ['Jack', 18]]
df = pd.DataFrame(data, columns=['Name',
'Age'])
print(df)
```

Пример: Создание фрейма данных из списка списков

	Name	Age
0	Tom	20
1	Nick	21
2	Krish	19
3	Jack	18

Вывод

```
import numpy as np

data = np.array(['', 'Col1', 'Col2'],
                ['Row1', 1, 2], ['Row2', 3, 4])
df = pd.DataFrame(data=data[1:, 1:],
                  index=data[1:, 0], columns=data[0, 1:])
print(df)
```

Пример: Создание фрейма данных из массива NumPy

	Col1	Col2
Row1	1	2
Row2	3	4

Вывод

Пример: Создание фрейма данных из файла CSV

```
df = pd.read_csv('data.csv')  
print(df)
```

```
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': [10, 20, 30], 'B': [40, 50, 60]})

# Сложение
result= df1 + df2
print(result)
```

Пример: Арифметические операции

	A	B
0	11	44
1	22	55
2	33	66

Вывод

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# Сумма
print(df.sum())
```

```
# Среднее
print(df.mean())
```

Пример: Агрегация

```
A      6
B     15
dtype: int64
A      2.0
B      5.0
dtype: float64
```

Вывод

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# Доступ к первому ряду
print(df.iloc[0])
```

```
# Нарезка строк
print(df.iloc[1:3])
```

Пример: Позиционное индексирование

```
A      1
B      4
Name: 0, dtype: int64
   A  B
1  2  5
2  3  6
```

Вывод

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=['row1', 'row2', 'row3'])

# Доступ к ряду по метке
print(df.loc['row2'])

# Нарезка строк по меткам
print(df.loc['row1':'row2'])
```

Пример: Индексирование на основе меток

```
A      2
B      5
Name: row2, dtype: int64

   A  B
row1 1  4
row2 2  5
```

Вывод

Атрибуты фреймов данных



`df.shape`: Возвращает кортеж, представляющий размерность фрейма данных.



`df.columns`: Возвращает метки столбцов DataFrame.



`df.index`: Возвращает метки строк фрейма DataFrame.

Методы фреймов данных



`df.head(n)`: Возвращает первые `n` строк фрейма `DataFrame`.



`df.tail(n)`: Возвращает последние `n` строк фрейма `DataFrame`.



`df.describe()`: Генерирует описательную статистику фрейма `DataFrame`.



`df.info()`: Предоставляет краткую информацию о `DataFrame`.



`df.drop(columns)`: Удаляет указанные столбцы из `DataFrame`.



`df.rename(columns)`: Переименовывает столбцы в `DataFrame`.

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# Форма фрейма данных
print(df.shape)
```

```
# Имена столбцов
print(df.columns)
```

```
# Первые две строки
print(df.head(2))
```

```
# Описательная статистика
print(df.describe())
```

```
(3, 2)
Index(['A', 'B'], dtype='object')
   A  B
0  1  4
1  2  5

   A  B
count  3.0  3.0
mean   2.0  5.0
std     1.0  1.0
min     1.0  4.0
25%     1.5  4.5
50%     2.0  5.0
75%     2.5  5.5
max     3.0  6.0
```

Пример: Использование атрибутов и методов

Вывод



Очистка данных

Это важнейший этап процесса анализа данных, включающий выявление и устранение ошибок, несоответствий и недостающих значений в наборе данных. Гарантирует, что данные точны, надежны и готовы к анализу или моделированию.

Очистка данных в Pandas

Опускание пропущенных значений

```
df_cleaned = df.dropna()
```

Заполнение пропущенных значений

```
df_filled = df.fillna(df.mean())
```

Удаление дубликатов

```
df_no_duplicates = df.drop_duplicates()
```

Исправление типов данных

```
df['column'] = df['column'].astype('int')
```

Переименование столбцов

```
df.rename(columns={'old_name': 'new_name'}, inplace=True)
```

Исследовательский анализ данных (EDA)

```
print(df.info())  
print(df.describe())  
print(df.head())
```

```
import pandas as pd

data = {'A': [1, 2, 3, None, 5], 'B': [None,
2, 3, 4, 5], 'C': [1, 2, None, None, 5]}
df = pd.DataFrame(data)
df_cleaned = df.dropna()
print(df_cleaned)
```

Пример: Удаление строк с пропущенными значениями

	A	B	C
1	2.0	2.0	2.0
4	5.0	5.0	5.0

Вывод

```
df_filled = df.fillna(df.mean())
print(df_filled)
```

Пример: Заполнение пропущенных значений средним значением

	A	B	C
0	1.00	3.5	1.000000
1	2.00	2.0	2.000000
2	3.00	3.0	2.666667
3	2.75	4.0	2.666667
4	5.00	5.0	5.000000

Вывод


```
df_no_duplicates = df.drop_duplicates()
print(df_no_duplicates)
```

Пример: Удаление дублирующихся строк

```
df.rename(columns={'A': 'Column_A', 'B':
'Column_B'}, inplace=True)
print(df)
```

Пример: Переименование столбцов



Слияние и объединение

Это операции при работе с данными, позволяющие объединить несколько фреймов данных на основе общих столбцов или индексов.

Экспресс-опрос

?

Что такое merge()?



merge()

Это функция, которая является основным инструментом для объединения DataFrames. Поддерживает различные типы объединений, включая внутренние, внешние, левые и правые объединения.

```
import pandas as pd

df1 = pd.DataFrame({'a': ['a1', 'a2', 'a3'],
                    'b': ['b1', 'b2', 'b3']})
df2 = pd.DataFrame({'a': ['a1', 'a2', 'a4'],
                    'c': ['c1', 'c2', 'c4']})

merged_df = pd.merge(df1, df2, on='a')
print(merged_df)
```

Пример: Слияние фреймов данных

	a	b	c
0	a1	b1	c1
1	a2	b2	c2

Вывод

Типы объединений в merge()

Внутреннее соединение

```
pd.merge(df1, df2, how='inner', on='a')
```

Левое объединение

```
pd.merge(df1, df2, how='left', on='a')
```

Правое объединение

```
pd.merge(df1, df2, how='right', on='a')
```

Внешнее объединение

```
pd.merge(df1, df2, how='outer', on='a')
```

```
df2 = df2.rename(columns={'a': 'a_'})
merged_df = pd.merge(df1, df2, left_on='a',
right_on='a_')
print(merged_df)
```

Пример: Указание столбцов для слияния

	a	b	a_	c
0	a1	b1	a1	c1
1	a2	b2	a2	c2

Вывод

Экспресс-опрос

?

Что такое join()?



join()

Это метод, метод экземпляра, который объединяет фреймы DataFrames на основе их индексов. Это более простой и эффективный способ объединения DataFrames по сравнению с `merge()`, когда вы работаете с индексами.

```
df1 = pd.DataFrame({'a': ['a1', 'a2', 'a3']},
index=['i1', 'i2', 'i3'])
df2 = pd.DataFrame({'b': ['b1', 'b2', 'b3']},
index=['i1', 'i2', 'i4'])

joined_df = df1.join(df2, how='left')
print(joined_df)
```

Пример: Базовое использование join()

	a	b
i1	a1	b1
i2	a2	b2
i3	a3	NaN

Вывод

Типы объединений в join()

Left Join

```
df1.join(df2, how='left')
```

Right Join

```
df1.join(df2, how='right')
```

Внутреннее соединение

```
df1.join(df2, how='inner')
```

Внешнее соединение

```
df1.join(df2, how='outer')
```

```
df1 = pd.DataFrame({'a': ['a1', 'a2', 'a3']})
df2 = pd.DataFrame({'b': ['b1', 'b2', 'b3']})

concatenated_df = pd.concat([df1, df2],
axis=1)
print(concatenated_df)
```

Пример: Базовое использование функции
concat()

	a	b
0	a1	b1
1	a2	b2
2	a3	b3

Вывод

```
concatenated_df = pd.concat([df1, df2],
keys=['df1', 'df2'])
print(concatenated_df)
```

Пример: Конкатенация с ключами

		a	b
df1	0	a1	NaN
	1	a2	NaN
	2	a3	NaN
df2	0	NaN	b1
	1	NaN	b2
	2	NaN	b3

Вывод



Группировка данных и создание сводных таблиц

Это операции при работе с данными, позволяющие объединить несколько фреймов данных на основе общих столбцов или индексов.

```
import pandas as pd

# Образец данных
data = {
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],
    'Value': [10, 20, 30, 40, 50, 60]
}
df = pd.DataFrame(data)

# Группируем по "Category" и вычисляем сумму "Value"
grouped_df = df.groupby('Category').sum()
print(grouped_df)
```

Пример: Базовое использование функции groupby()

Category	Value
A	90
B	120

Вывод

```
# Группировка по 'Category' и вычисление
нескольких агрегированных значений
grouped_df =
df.groupby('Category').agg({'Value': ['sum',
'mean', 'max']})
print(grouped_df)
```

Пример: Расширенная группировка

	Value		
	sum	mean	max
Category			
A	90	30.0	50
B	120	40.0	60

Вывод


```
# Образец данных
data = {
    'A': ['foo', 'foo', 'foo', 'bar', 'bar', 'bar'],
    'B': ['one', 'one', 'two', 'two', 'one', 'one'],
    'C': ['маленький', 'большой', 'маленький',
          'маленький', 'большой', 'маленький'],
    'D': [1, 2, 2, 3, 3, 4]
}
df = pd.DataFrame(data)

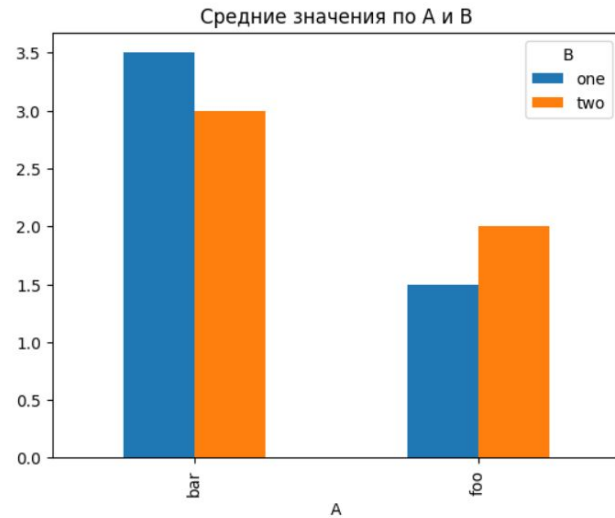
# Создание сводной таблицы
pivot_table = pd.pivot_table(df, values='D', index=['A',
'B'], columns=['C'], aggfunc='sum', fill_value=0)
print(pivot_table)
```

Пример: Создание сводных таблиц с помощью функции `pivot_table()`

C		большой	маленький
A	B		
bar	one	3	4
	two	0	3
foo	one	2	1
	two	0	2

Вывод

```
# Построение графиков по таблице pivot
pivot_table = pd.pivot_table(df, values='D',
index='A', columns='B', aggfunc='mean')
pivot_table.plot(kind='bar', title='Средние
значения по A и B')
```



Пример: Построение графиков в таблицах
Pivot

Вывод

```

pivot_table = pd.pivot_table(df,
values=['D'], index=['A', 'C'], aggfunc={'D':
['mean', 'sum', 'max']})
print(pivot_table)

```

Пример с несколькими агрегациями

A	C	D		
		max	mean	sum
bar	большой	3	3.0	3
	маленький	4	3.5	7
foo	большой	2	2.0	2
	маленький	2	1.5	3

Вывод



ВОПРОСЫ



Экспресс-опрос



Что такое Matplotlib?



Matplotlib

Это универсальная библиотека для создания широкого спектра статических, анимированных и интерактивных графиков на Python.



Линейные графики

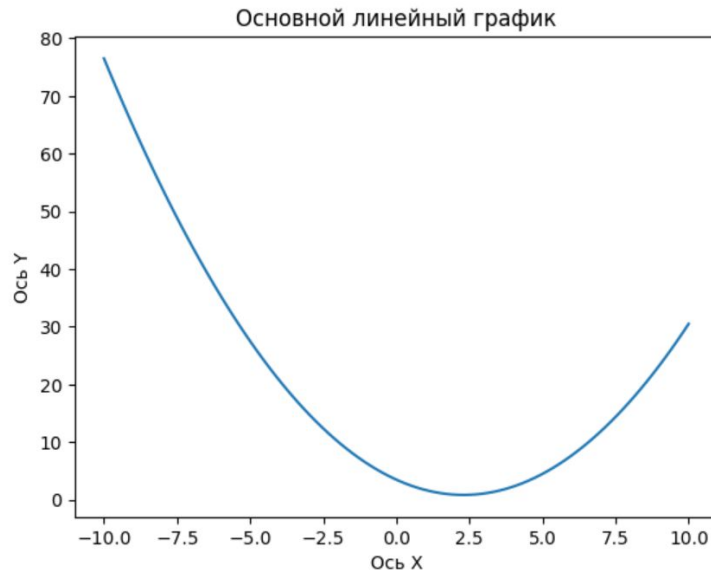
Используются для визуализации тенденций изменения данных за интервалы времени.

```
import matplotlib.pyplot as plt
import numpy as np

# Создаем квадратичную кривую
x = np.linspace(-10, 10, 100)
y = 3.5 - 2.3 * x + 0.5 * x ** 2

# Строим график данных
plt.plot(x, y)
plt.xlabel('Ось X')
plt.ylabel('Ось Y')
plt.title('Основной линейный график')
plt.show()
```

Создание базового линейного графика в Matplotlib



Результат



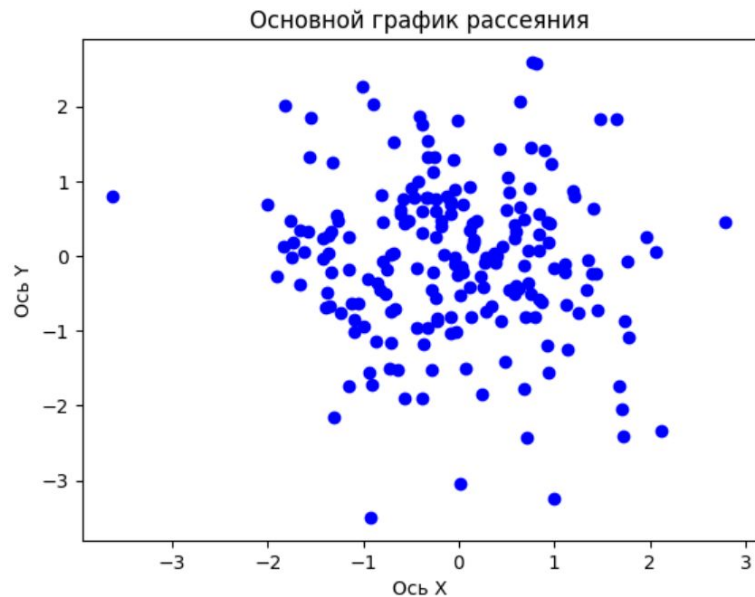
Графики рассеяния

Используются для отображения значений, как правило, двух переменных для набора данных.

```
# Генерируем случайные данные
x = np.random.randn(200)
y = np.random.randn(200)

# Создаём диаграмму рассеяния
plt.scatter(x, y, color='blue', marker='o')
plt.xlabel('Ось X')
plt.ylabel('Ось Y')
plt.title('Основной график рассеяния')
plt.show()
```

Создание графика рассеяния



Результат



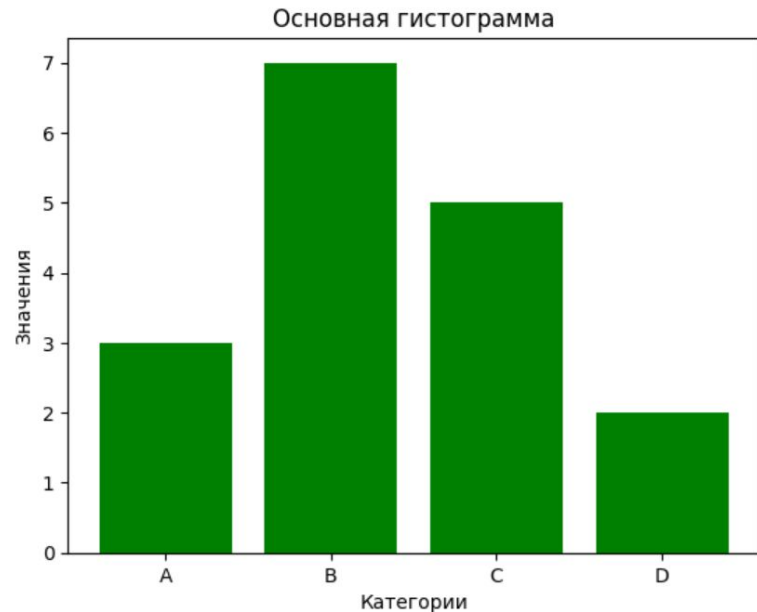
Столбчатые диаграммы

Используются для сравнения величин, относящихся к разным категориям.

```
# Определяем категории и значения
categories = ['A', 'B', 'C', 'D']
values = [3, 7, 5, 2]

# Создаём гистограмму
plt.bar(categories, values, color='green')
plt.xlabel('Категории')
plt.ylabel('Значения')
plt.title('Основная гистограмма')
plt.show()
```

Создание столбчатой диаграммы



Результат



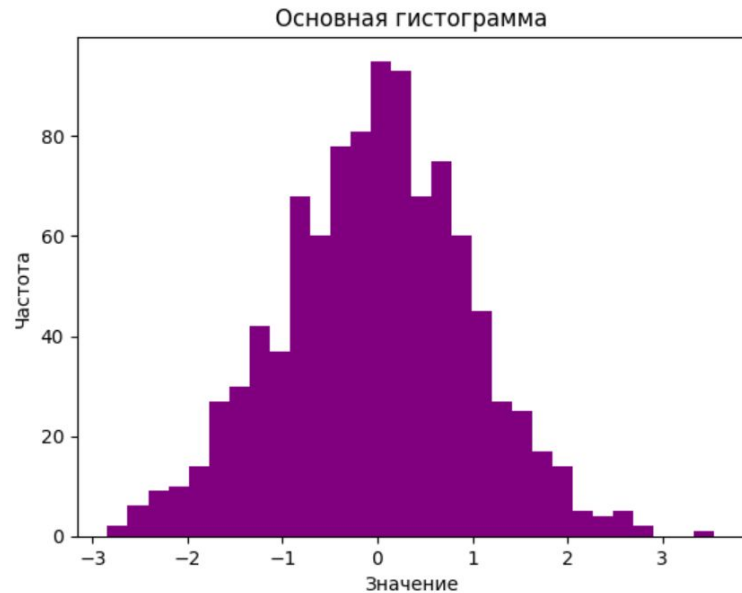
Гистограммы

Используются для представления распределения набора данных.

```
# Генерируем случайные данные
data = np.random.randn(1000)

# Создаём гистограмму
plt.hist(data, bins=30, color='purple')
plt.xlabel('Значение')
plt.ylabel('Частота')
plt.title('Основная гистограмма')
plt.show()
```

Создание гистограммы



Результат

Заголовки

```
import matplotlib.pyplot as plt
import numpy as np

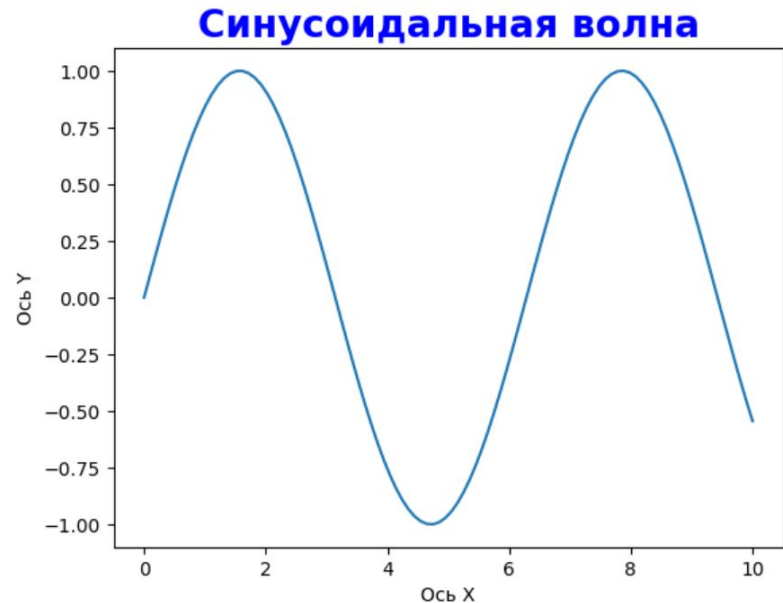
# Данные выборки
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Построение графика данных
plt.plot(x, y)

# Настройка заголовка
plt.title('Синусоидальная волна', fontsize=20,
fontweight='bold', color='blue')
plt.xlabel('Ось X')
plt.ylabel('Ось Y')

# Отображение графика
plt.show()
```

Добавление заголовка



Результат

Метки

```
import matplotlib.pyplot as plt
import numpy as np

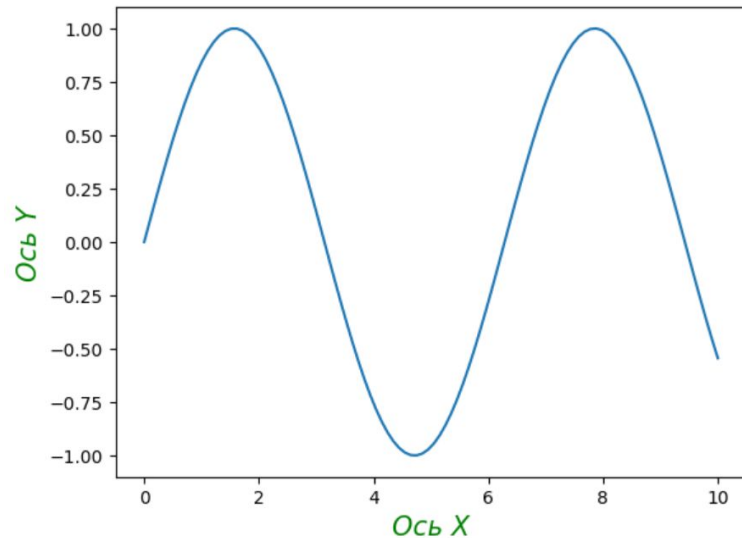
# Данные выборки
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Построение графика данных
plt.plot(x, y)

# Настройка меток
plt.xlabel('Ось X', fontsize=15,
           fontstyle='italic', color='green')
plt.ylabel('Ось Y', fontsize=15,
           fontstyle='italic', color='green')

plt.show()
```

Добавление меток



Результат

Легенды

```
# Примерные данные
y1 = np.sin(x)
y2 = np.cos(x)

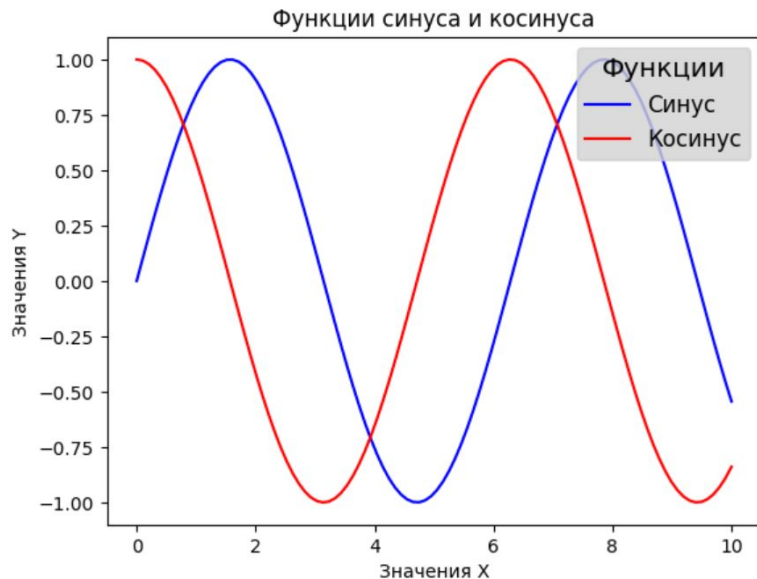
# Построение графика с метками и цветами
plt.plot(x, y1, label='Синус', color='blue')
plt.plot(x, y2, label='Косинус', color='red')

# Настройка легенды
plt.legend(loc='upper right', fontsize='large',
title='Функции', title_fontsize='x-large',
facecolor='lightgrey', labelcolor='black')

# Добавление деталей
plt.title('Функции синуса и косинуса')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

# Отображение графика
plt.show()
```

Добавление легенды



Результат

Цвета

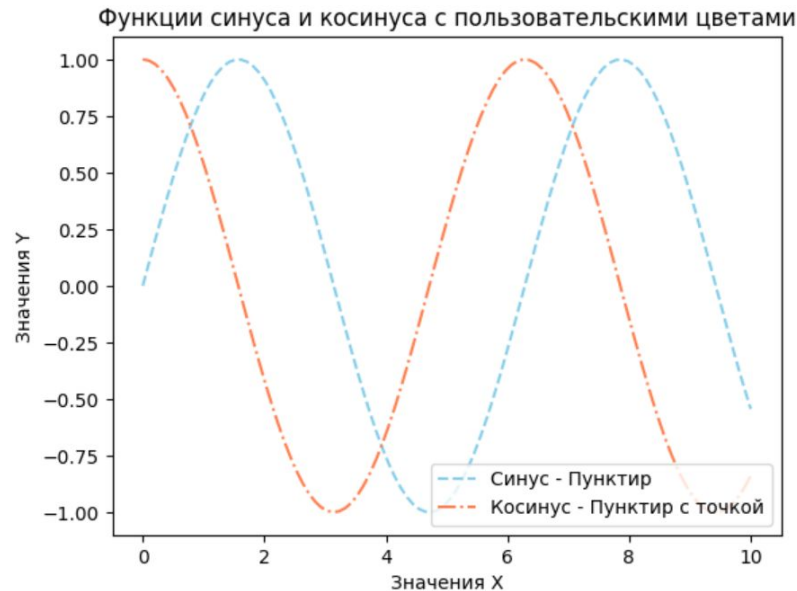
```
# Построение графиков с использованием
пользовательских цветов
plt.plot(x, y1, label='Синус', color='skyblue',
linestyle='--')
plt.plot(x, y2, label='Косинус', color='coral',
linestyle='-.')

# Настройка легенды
plt.legend(labels=['Синус - Пунктир', 'Косинус -
Пунктир с точкой'], loc='lower right',
fontsize='medium')

# Добавление деталей
plt.title('Функции синуса и косинуса с
пользовательскими цветами')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

# Отображение графика
plt.show()
```

Добавление цветов



Результат



Подграфики

Это функция в Matplotlib, позволяющая создавать фигуры, содержащие несколько графиков, которые могут быть расположены в различных макетах.

Функция plt.subplots()

```
import matplotlib.pyplot as plt
import numpy as np

# Создаем примерные данные
x = np.linspace(0, 2 * np.pi, 400)
y = np.sin(x ** 2)

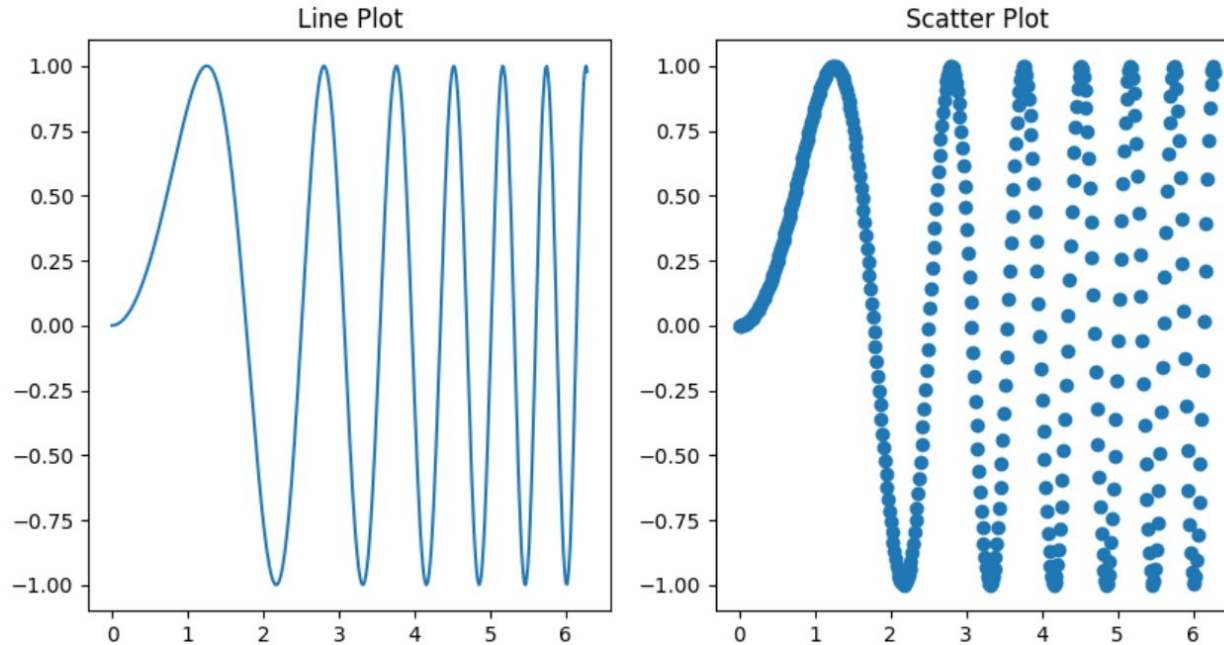
# Создайте рисунок с двумя подплощадками
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))

# Нанесите данные на первый подграфик
ax1.plot(x, y)
ax1.set_title('Line Plot')

# Постройте данные на втором подграфике
ax2.scatter(x, y)
ax2.set_title('Scatter Plot')

# Отобразите рисунок
plt.show()
```

Функция plt.subplots()





GridSpec

Позволяет гибко позиционировать подграфики в сетке. Подграфики могут охватывать несколько строк или столбцов.

GridSpec

```
from matplotlib.gridspec import GridSpec

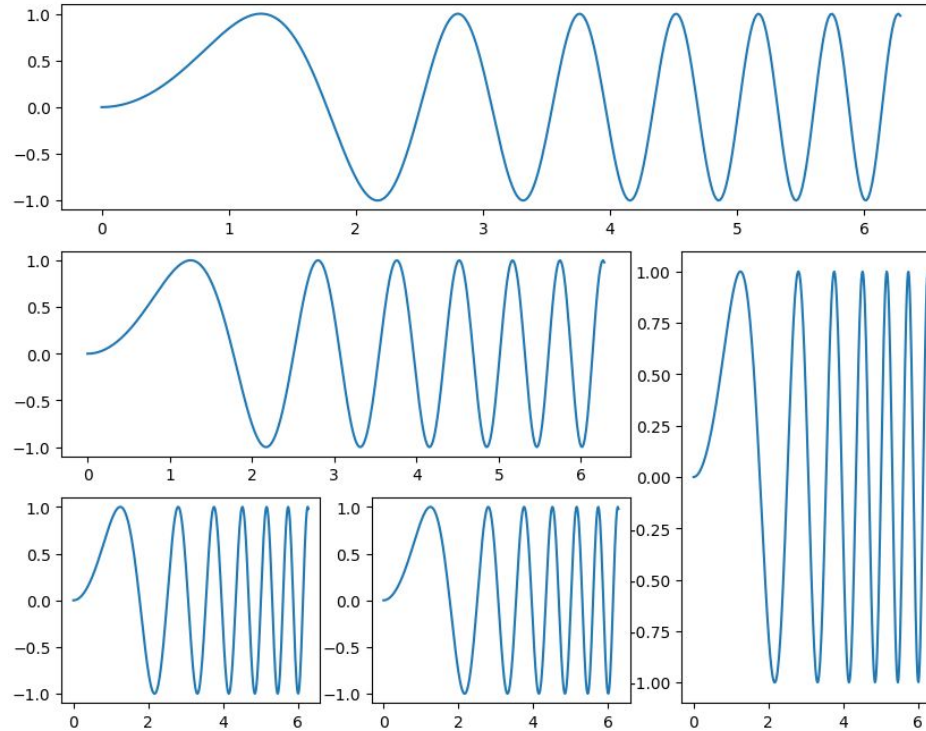
fig = plt.figure(figsize=(10, 8))
gs = GridSpec(3, 3, figure=fig)

ax1 = fig.add_subplot(gs[0, :])
ax2 = fig.add_subplot(gs[1, :-1])
ax3 = fig.add_subplot(gs[1:, -1])
ax4 = fig.add_subplot(gs[-1, 0])
ax5 = fig.add_subplot(gs[-1, -2])

# Построение данных на подграфиках
ax1.plot(x, y)
ax2.plot(x, y)
ax3.plot(x, y)
ax4.plot(x, y)
ax5.plot(x, y)

plt.show()
```

GridSpec



Экспресс-опрос

?

Что такое subplot_mosaic?



`subplot_mosaic`

Позволяет задавать структуру фигуры с помощью форматированных строк.

subplot_mosaic

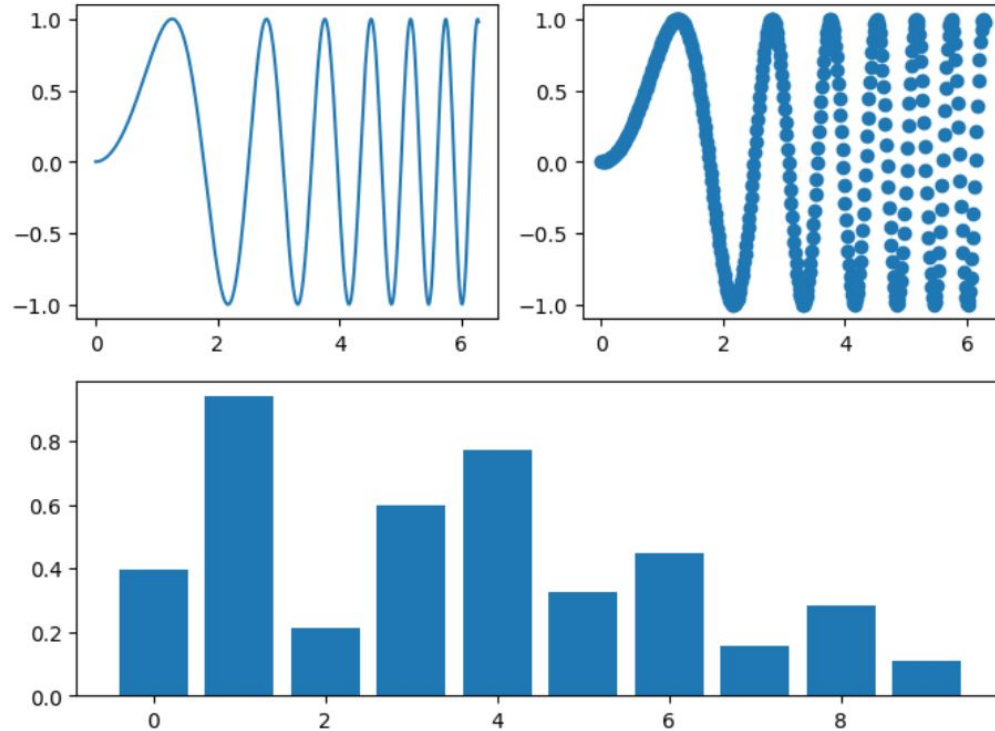
```
layout = """
AB
CC
"""

fig, axes = plt.subplot_mosaic(layout, figsize=(8, 6))

axes['A'].plot(x, y)
axes['B'].scatter(x, y)
axes['C'].bar(np.arange(10), np.random.rand(10))

plt.show()
```

subplot_mosaic





Seaborn

Это библиотека Python, построенная поверх Matplotlib и предназначенная для создания привлекательных и информативных статистических графиков.



Графики распределения

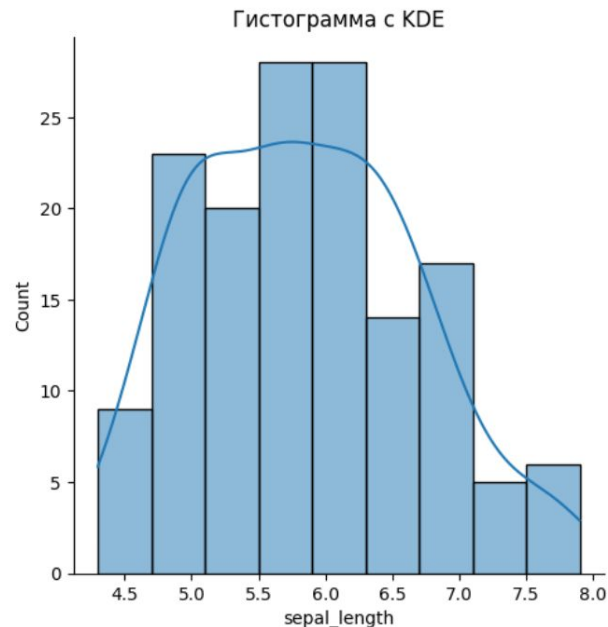
Используются для визуализации распределения набора данных.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Загружаем данные выборки
data = sns.load_dataset("iris")

# Гистограмма с помощью KDE
sns.displot(data['sepal_length'], kde=True)
plt.title('Гистограмма с KDE')
plt.show()
```

Создание графика распределения



Результат

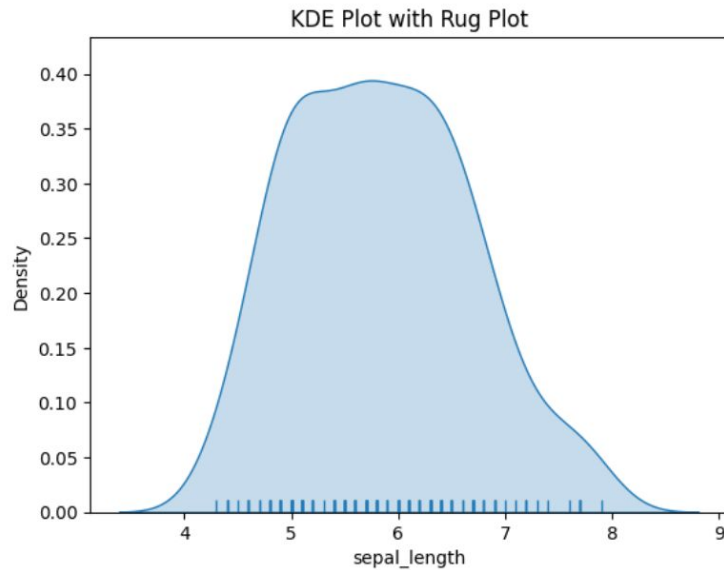


Rug графики

Добавляют небольшие вертикальные линии в каждой точке данных вдоль оси x, обеспечивая визуальное представление распределения данных.


```
sns.kdeplot(data['sepal_length'], fill=True)
sns.rugplot(data['sepal_length'])
plt.title('KDE Plot with Rug Plot')
plt.show()
```

Создание rug графика



Результат

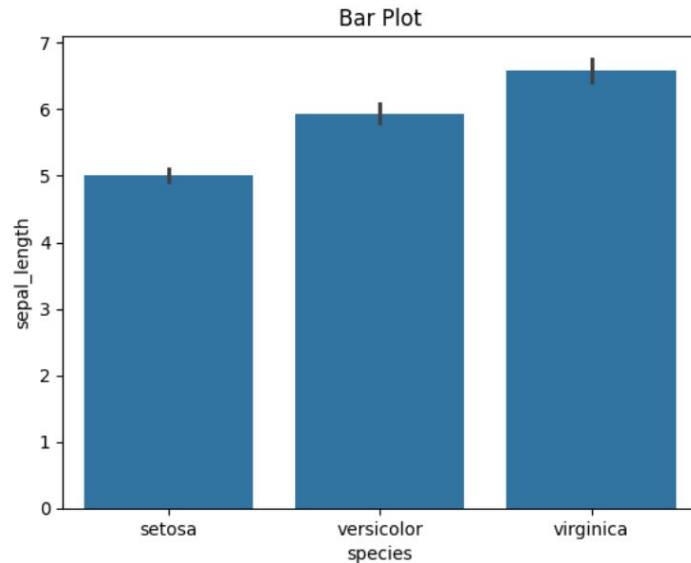


Категориальные графики

Используются для визуализации данных, в которых одна или несколько переменных являются категориальными.

```
# Гистограмма
sns.barplot(x='species', y='sepal_length',
data=data)
plt.title('Bar Plot')
plt.show()
```

Создание категориального графика



Результат

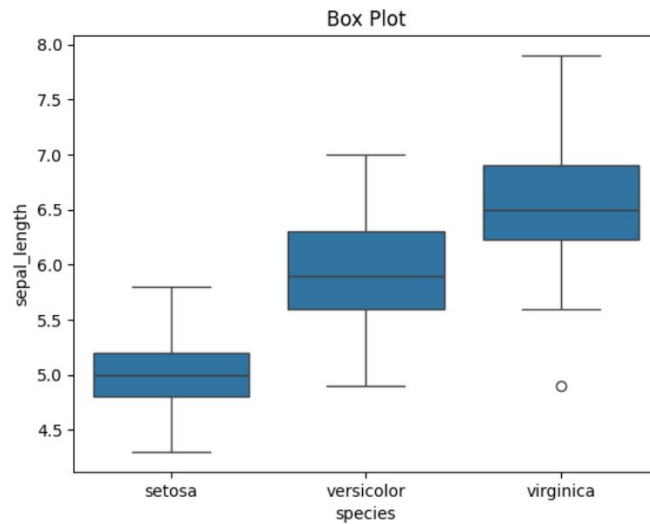


Графики ящик с усами

Отображают распределение непрерывной переменной для каждой категории категориальной переменной.

```
# Коробчатый график
sns.boxplot(x='species', y='sepal_length',
data=data)
plt.title('Box Plot')
plt.show()
```

Создание графика ящик с усами



Результат

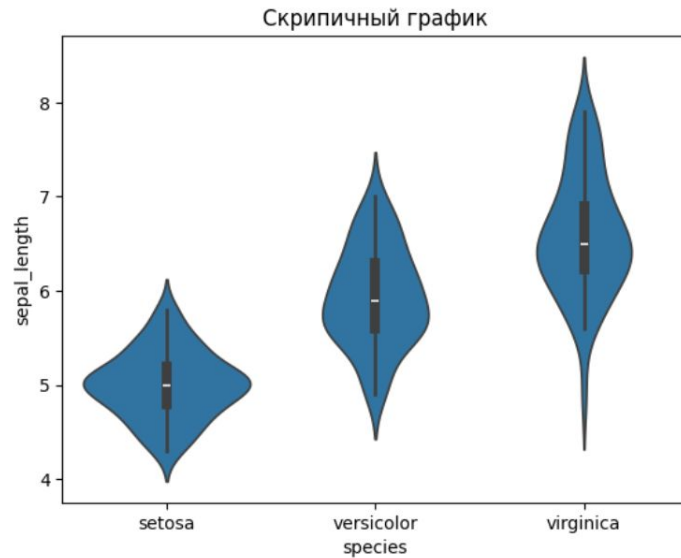


Скрипичные графики

Полезны для сравнения распределения непрерывной переменной по разным категориям.

```
# Скрипичный график
sns.violinplot(x='species', y='sepal_length',
data=data)
plt.title('Скрипичный график')
plt.show()
```

Создание скрипичного графика



Результат

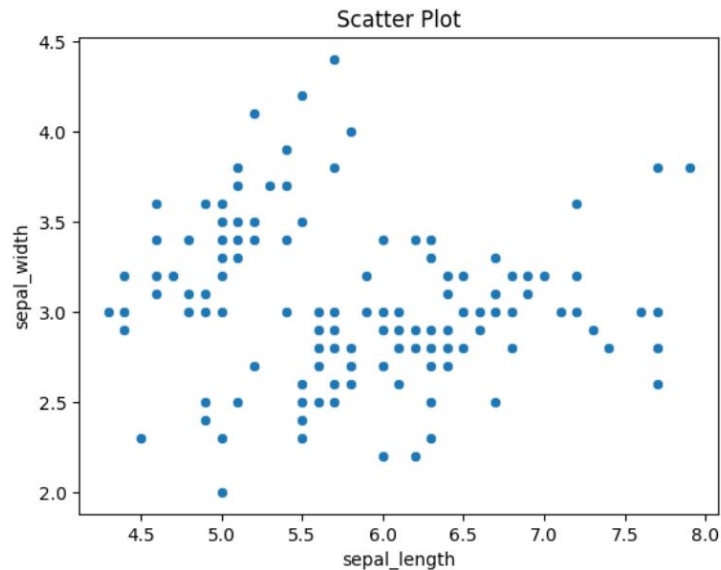


Реляционные графики

Используются для визуализации взаимосвязи между двумя или более переменными.


```
# Диаграмма рассеяния
sns.scatterplot(x='sepal_length',
y='sepal_width', data=data)
plt.title('Scatter Plot')
plt.show()
```

Создание реляционного графика



Результат

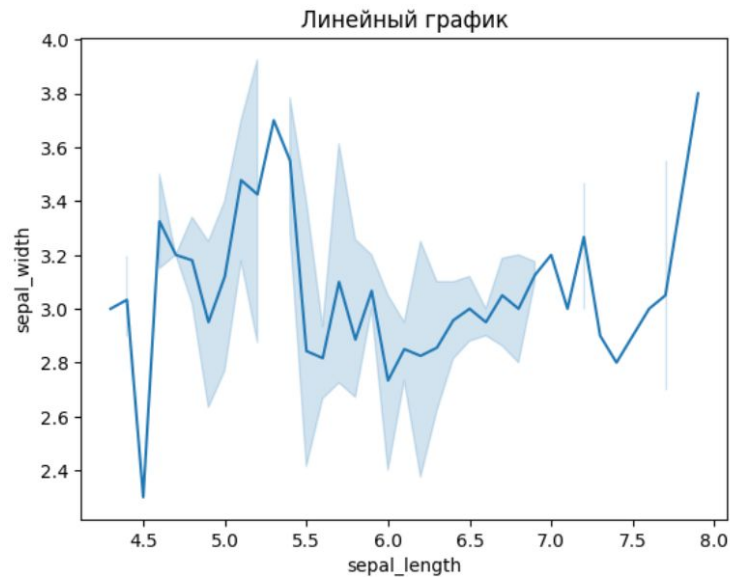


Линейные графики

Полезны для визуализации тенденций с течением времени или другой непрерывной переменной.

```
# Линейный график
sns.lineplot(x='sepal_length',
y='sepal_width', data=data)
plt.title('Линейный график')
plt.show()
```

Создание линейного графика



Результат



Реляционные графики

Полезны для изучения и понимания взаимосвязей между несколькими переменными в наборе данных.



Парные графики

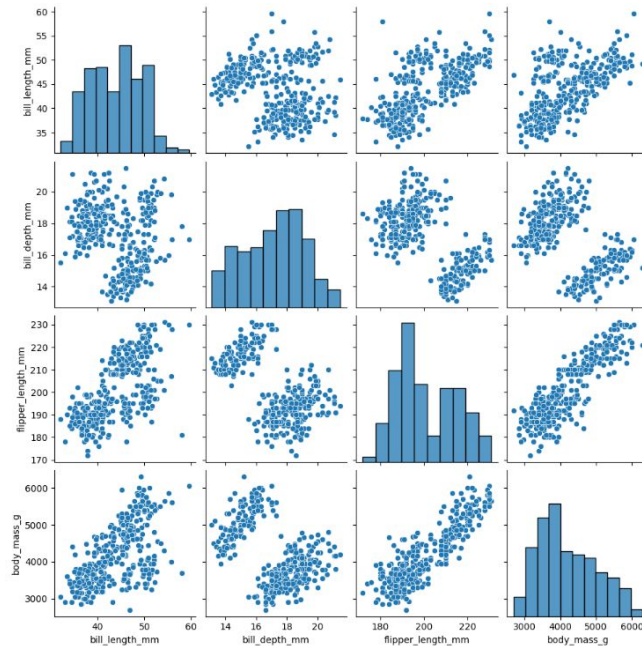
Используются для визуализации парных отношений в наборе данных. Полезны для изучения взаимосвязей между несколькими переменными одновременно.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Загрузка данных выборки
penguins = sns.load_dataset("penguins")

# Создайте парный график
sns.pairplot(penguins)
plt.show()
```

Создание парных графиков



Результат

Парные графики

```
# Парный график с оттенком
sns.pairplot(penguins, hue="species")
plt.show()
```

```
# Парный график с различными видами графиков
sns.pairplot(penguins, kind="kde")
plt.show()
```

```
# Парный график с пользовательскими маркерами и размерами
sns.pairplot(penguins, hue="species", markers=["o", "s", "D"], height=2.5)
plt.show()
```



Тепловые карты

Полезны для визуализации корреляционной матрицы между несколькими переменными, позволяя с первого взгляда определить высокоррелированные или обратнокоррелированные переменные.

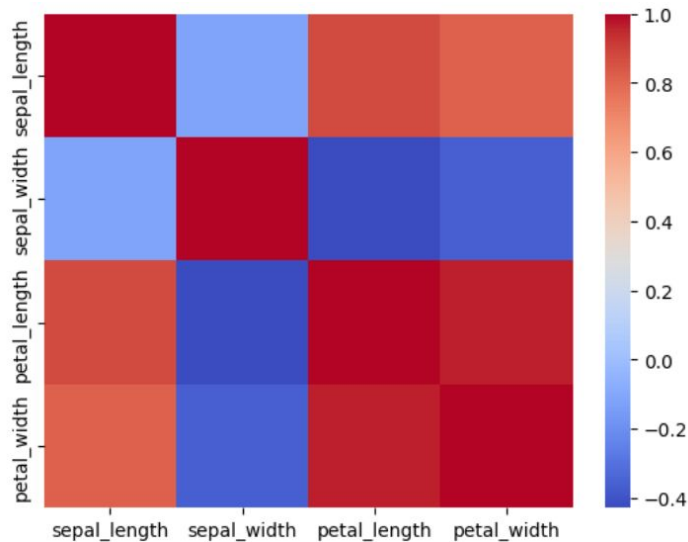

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Загрузка данных выборки
data = sns.load_dataset("iris")
data = data[["sepal_length", "sepal_width",
"petal_length", "petal_width"]]

# Рассчитайте корреляционную матрицу
correlation_matrix = data.corr()

# Создайте тепловую карту
sns.heatmap(correlation_matrix,
cmap='coolwarm')
plt.show()
```

Создание тепловых графиков



Результат



ВОПРОСЫ





Статистика

Это отрасль математики, которая занимается сбором, анализом, интерпретацией, представлением и организацией данных.



Описательная статистика

Обобщает и описывает основные характеристики набора данных. Они предоставляют простые сводки о выборке и показателях.

Основные компоненты описательной статистики

Меры центральной
тенденции

Среднее значение

Медиана

Мода

Меры
изменчивости
(дисперсии)

Диапазон

Дисперсия

Стандартное
отклонение

Частотное
распределение

Гистограммы

Столбчатые
диаграммы

Круговые
диаграммы

Форма
распределения

Перекоз

Экссесс

Примеры описательной статистики



Вычисление среднего тестового балла студентов в классе.



Использование гистограммы для отображения распределения возрастов в генеральной совокупности.



Обобщение результатов опроса с помощью среднего значения, медианы и моды.



Инференциальная статистика

Используется для того, чтобы делать выводы или обобщения о генеральной совокупности на основе выборки данных.

Основные компоненты инференциальной статистики



Примеры инференциальной статистики



Использование выборочного обследования для оценки среднего дохода населения.



Проведение t-теста, чтобы определить, существует ли значительная разница между средними показателями двух групп.



Проведение регрессионного анализа для прогнозирования будущих продаж на основе прошлых данных.



Меры центральной тенденции

Это статистические показатели, которые описывают центральную точку или типичное значение набора данных.

Среднее



Определение

Это сумма всех значений в наборе данных, деленную на количество значений.

Пример

```
import numpy as np

data = [1, 2, 3, 4, 5]
mean = np.mean(data)
print("Среднее:", mean)
```

Медиана



Определение

Это среднее значение набора данных, упорядоченное от наименьшего к наибольшему.

Пример

```
import numpy as np

data = [1, 2, 3, 4, 5]
median = np.median(data)
print("Медиана:", median)
```

Мода



Определение

Это значение, которое чаще всего встречается в наборе данных.

Пример

```
from scipy import stats

data = [1, 1, 2, 3, 4, 6, 18]
mode_result = stats.mode(data)
mode = mode_result.mode
print("Мода:", mode)
```

Пример



Вычисление среднего, медианы и моды с помощью Python SciPy.

Набор данных

```
data = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]
```

Вычисление среднего значения

```
import numpy as np

mean = np.mean(data)
print("Среднее:", mean)
```


Вычисление медианы

```
median = np.median(data)  
print("Медиана:", median)
```

Вычисление моды

```
from scipy import stats

mode_result = stats.mode(data)
mode = mode_result.mode
print("Мода:", mode)
```

Экспресс-опрос

?

Что такое меры дисперсии?



Меры дисперсии

Это статистические инструменты, используемые для описания разброса или изменчивости в наборе данных.

Диапазон



Определение

Это самая простая мера дисперсии.

Пример

```
data = [4, 8, 6, 5, 3, 2, 8, 9, 2, 5]  
range_value = max(data) - min(data)  
print("Диапазон:", range_value)
```

Вариация



Определение

Это среднее квадратичное отклонение каждой точки данных от среднего значения.

Пример

```
import numpy as np

data = [4, 8, 6, 5, 3, 2, 8, 9, 2, 5]
variance = np.var(data)
print("Вариация:", variance)
```

Стандартное отклонение



Определение

Это квадратный корень из дисперсии.

Пример

```
import numpy as np

data = [4, 8, 6, 5, 3, 2, 8, 9, 2, 5]
std_deviation = np.std(data)
print("Стандартное отклонение:",
      std_deviation)
```

Пример



Вычисление диапазона, дисперсии и стандартного отклонения с помощью Python

Набор данных

```
data = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]
```

Вычисление диапазона

```
range_value = max(data) - min(data)  
print("Диапазон:", range_value)
```

Вычисление дисперсии

```
variance = np.var(data)  
print("Вариация:", variance)
```

Вычисление стандартного отклонения

```
std_deviation = np.std(data)  
print("Стандартное отклонение:", std_deviation)
```



Проверка гипотез

Это статистический метод, используемый для того, чтобы сделать выводы о совокупности на основе выборочных данных.

Нулевая гипотеза



Определение

Это утверждение об отсутствии эффекта или различий, представляющее собой статус-кво или базовое предположение.

Пример

Если мы проверяем, является ли монета честной, нулевая гипотеза может состоять в том, что вероятность выпадения решек равна 0,5.

Альтернативная гипотеза



Определение

Это утверждение, противоречащее нулевой гипотезе. Она представляет собой эффект или разницу, которую исследователь стремится обнаружить.

Пример

Для теста на честность монеты альтернативная гипотеза может заключаться в том, что вероятность выпадения решек не равна 0,5.

Шаги при проверке гипотез

1. Сформулируйте нулевую и альтернативную гипотезы.
2. Выберите уровень значимости (α), - это вероятность отвергнуть нулевую гипотезу, если она верна. Обычно выбирают 0,05, 0,01 и 0,10.
3. Выберите статистический тест в зависимости от типа данных и гипотезы. К распространенным тестам относятся t-тесты, тесты хи-квадрат, ANOVA и регрессионный анализ.
4. Соберите данные, которые будут анализироваться в тесте.
5. На основе собранных данных и выбранного теста рассчитайте тестовую статистику, которая отражает, насколько наблюдаемые данные отклоняются от нулевой гипотезы.
6. Определите р-значение - это вероятность того, что результаты теста будут не менее экстремальными, чем наблюдаемые результаты, при условии, что нулевая гипотеза верна.
7. Сравните р-значение с выбранным уровнем значимости:
 - Если $p\text{-значение} \leq \alpha$: Отклоните нулевую гипотезу.
 - Если $p\text{-значение} > \alpha$: Не отвергайте нулевую гипотезу.
8. Представьте результаты проверки гипотез, включая тестовую статистику, р-значение и вывод о гипотезах.

Экспресс-опрос

?

Что такое t-тест?



t-тест

Используется для сравнения средних значений двух групп.

Пример t-теста

```
from scipy import stats

# Данные выборки
data1 = [1, 2, 3, 4, 5]
data2 = [6, 7, 8, 9, 10]

# Выполните t-тест
t_statistic, p_value = stats.ttest_ind(data1, data2)

print('t-статистика:', t_statistic)
print('p-value:', p_value)
```

Экспресс-опрос



Что такое тест хи-квадрат?



Тест хи-квадрат

Используется для определения наличия значимой связи между двумя категориальными переменными.

Пример теста хи-квадрат

```
from scipy.stats import chi2_contingency
import numpy as np

# Таблица контингентности
data = np.array([[207, 282, 241], [234, 242, 232]])

# Выполните тест хи-квадрат
chi2, p, dof, expected = chi2_contingency(data)

print('Статистика хи-квадрат:', chi2)
print('p-значение:', p)
print('Степени свободы:', dof)
print('Ожидаемые частоты:', expected)
```



Доверительный интервал

Это диапазон значений, полученный на основе выборочных данных, который, скорее всего, содержит истинное значение неизвестного параметра генеральной совокупности.

Ключевые понятия



Уровень доверия



Предел погрешности

Пример



Вычисление доверительных интервалов с помощью Python

Импорт библиотек

```
import numpy as np  
from scipy import stats
```

Определение выборочных данных

```
data = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]
```

Вычисление выборочного среднего и стандартного отклонения

```
sample_mean = np.mean(data)
sample_std = np.std(data, ddof=1) # ddof=1 для выборочного стандартного отклонения
n = len(data)
```

Определение Z-значения для 95-процентного уровня доверия

```
confidence_level = 0,95
alpha = 1 - confidence_level
z_value = stats.norm.ppf(1 - alpha/2)
```

Вычисление предела ошибки

```
margin_of_error = z_value * (sample_std / np.sqrt(n))
```

Расчет доверительного интервала

```
confidence_interval = (sample_mean - margin_of_error, sample_mean + margin_of_error)  
print("95% доверительный интервал:", confidence_interval)
```



ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикумов 19 и 20.



Задание 1

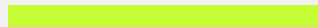
Создайте numpy-массив из 100 случайных значений. Преобразуйте его в матрицу 10x10. Вычислите определитель этой матрицы и обратную матрицу. Транспонируйте исходную матрицу. Перемножьте двумя способами (поэлементно и матричным умножением) транспонированную матрицу и обратную к исходной. Любую из матриц преобразуйте в датафрейм pandas. Дайте столбцам названия "A", "B", "C"... Удалите из этого датафрейма последние две строки.

Задание 2



Рассчитайте среднее значение и стандартное отклонение каждого признака для каждого вида в наборе данных Iris.

Задание 3



Рассчитайте среднюю сумму чаевых для каждой комбинации дня и времени (обед/ужин) в наборе данных Tips.

Задание 4

Рассчитайте ежегодный темп роста числа пассажиров в наборе данных Flights.

Задание 5



Рассчитайте коэффициент выживаемости для каждого сочетания класса и пола в наборе данных Titanic.

Задание 6



Нормализуйте признаки (sepal_length, sepal_width, petal_length, petal_width) с помощью Z-score нормализации в наборе данных Iris.

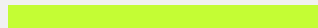
Задание 7

Рассчитайте диапазон (max - min) каждого признака для каждого вида в наборе данных Iris.

Задание 8

Заполните недостающие значения в столбце "Возраст" медианным возрастом пассажиров того же класса и пола в наборе данных Titanic.

Задание 9



Рассчитайте общий счет и средний процент чаевых для каждого дня в наборе данных Tips.

Задание 1

Загрузите датасет "Пингвины" из seaborn и познакомьтесь с ним:

- Постройте диаграмму рассеяния по значениям `bill_length_mm` и `bill_depth_mm`.
- Добавьте цветовое выделение групп по столбцу `sex`.
- Постройте круговую диаграмму, показывающую распределение пингвинов датасета по островам.
- Постройте столбчатую диаграмму, которая показывает среднюю массу пингвинов для каждого вида и пола.

Задание 2



Создайте парный график набора данных "Пингвины" с графиками KDE по диагонали и пользовательскими маркерами для разных видов.

Задание 3



Создайте скрипичный график для набора данных `tips`, показывающий распределение `total_bill`, разделенное по полу и окрашенное по дням.

Задание 4

Создайте линейный график набора данных flights, показывающий количество пассажиров с течением времени. Пометьте график максимальными и минимальными значениями.

Задание 5



Создайте совместный график набора данных `trg`, показывающий связь между лошадиными силами и расходом топлива с помощью линии регрессии и KDE.

Задание 6

Создайте систему подграфиков с набором данных "Титаник", включающую:

- Гистограмму количества пассажиров по классам.
- График возраста пассажиров по классам.
- Диаграмму-скрипку стоимости проезда по классам.

Задание 7



Сравните среднее содержание алкоголя в двух сортах вина из набора данных Wine.

Задание 8



Проверьте, существует ли значимая связь между размером опухоли и ее классом в наборе данных Breast Cancer.

Задание 9



Постройте модель логистической регрессии для предсказания класса рака молочной железы на основе среднего радиуса и средней текстуры.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Подгоните полином к набору точек данных:

```
x = np.linspace(-10, 10, N)
y = 3 * x**3 + 2 * x**2 + x + np.random.normal(0, 100, N)
```

и постройте график результатов. Используйте функцию `np.polyfit`.

2. Создайте возрастные группы и рассчитайте коэффициент выживаемости для каждой комбинации возрастной группы и класса в наборе данных Titanic.

Домашнее задание

1. Создайте многопанельный график, используя набор данных Iris. График должен включать:
 - Диаграмма рассеяния длины чашелистика по сравнению с шириной чашелистика на первой панели.
 - График разброса длины лепестка по ширине лепестка на второй панели.
 Обе панели должны иметь одинаковые границы по оси x и оси y.
2. Создайте тепловую карту корреляционной матрицы набора данных Tips. Аннотируйте тепловую карту значениями корреляции и используйте пользовательскую палитру расходящихся цветов.
3. Проверьте, значительно ли отличается средняя длина лепестка у видов Iris setosa от 1,5 см.



ВОПРОСЫ



Заключение

